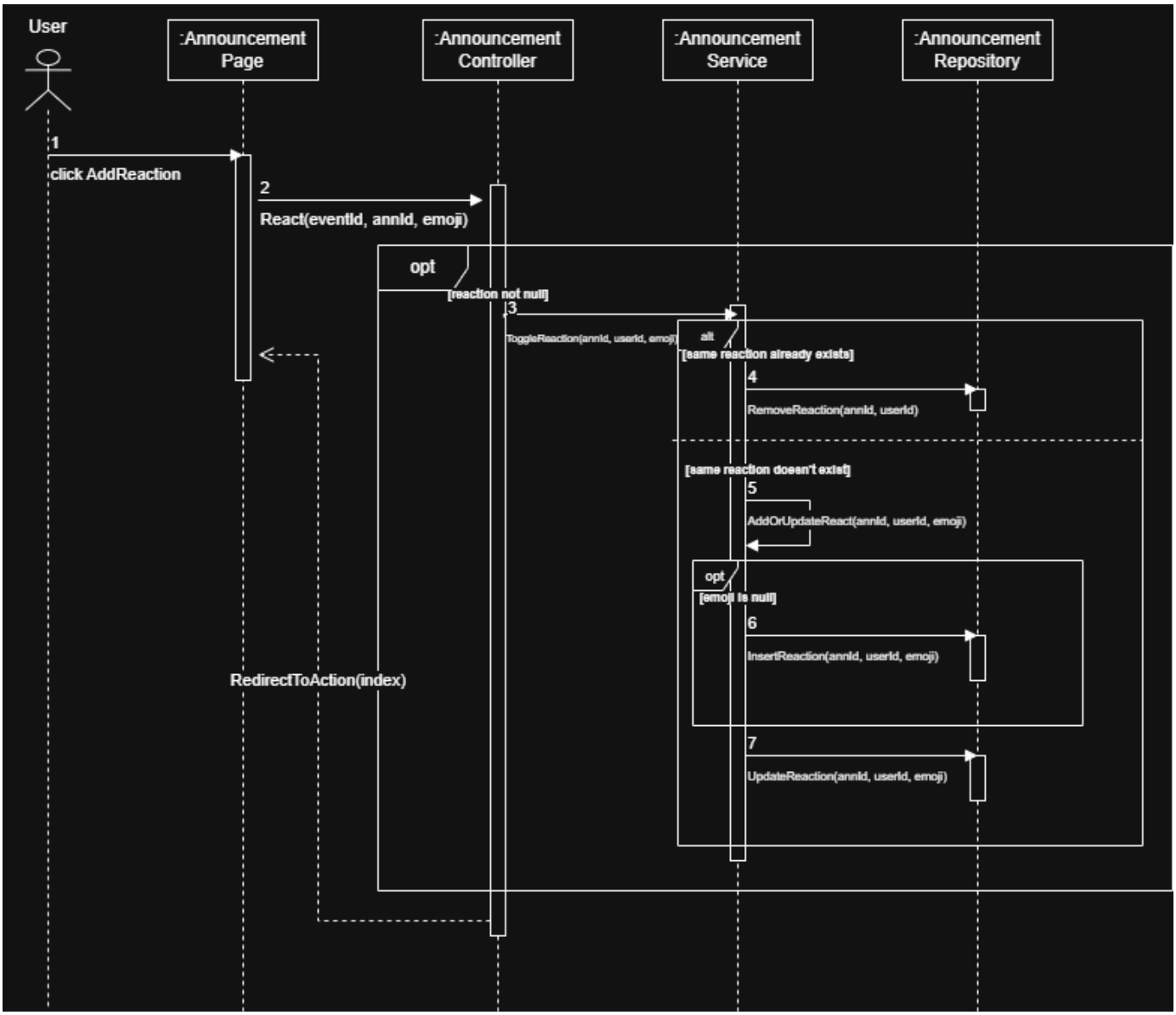
Diagram for adding a reaction by Denisa

Diagram



Source Code

View:

```
@if (announcement.ReactionGroups.Count > 0)
{
    <div class="d-flex flex-wrap gap-2 mt-2">
        @foreach (var reaction in announcement.ReactionGroups)
        {
            <span class="badge @(reaction.CurrentUserReacted ? "bg-primary" : "bg-secondary")">
                @reaction.Emoji @reaction.Count
            </span>
        }
    </div>
}

<div class="d-flex flex-wrap gap-1 mt-3">
    <form asp-action="React" method="post" class="d-flex flex-wrap gap-1">
        <input type="hidden" name="eventId" value="@Model.EventId" />
        <input type="hidden" name="announcementId" value="@announcement.Model.Id" />
        <button type="submit" name="emoji" value="👍" class="btn btn-sm btn-outline-secondary">👍</button>
        <button type="submit" name="emoji" value="❤️" class="btn btn-sm btn-outline-secondary">❤️</button>
        <button type="submit" name="emoji" value="😂" class="btn btn-sm btn-outline-secondary">😂</button>
        <button type="submit" name="emoji" value="😮" class="btn btn-sm btn-outline-secondary">😮</button>
        <button type="submit" name="emoji" value="😢" class="btn btn-sm btn-outline-secondary">😢</button>
        <button type="submit" name="emoji" value="🔥" class="btn btn-sm btn-outline-secondary">🔥</button>
    </form>
</div>
```

Controller:

```
[HttpPost]
0 references
public async Task<IActionResult> React(int eventId, int announcementId, string emoji)
{
    var currentUser = await _userService.GetCurrentUser();

    if (!string.IsNullOrWhiteSpace(emoji))
    {
        await _announcementService.ToggleReactionAsync(announcementId, currentUser.UserId, emoji);
    }

    return RedirectToAction(nameof(Index), new { eventId });
}
```

Service:

```
6 references
public async Task ToggleReactionAsync(int announcementId, Guid userId, string emoji)
{
    var existingEmoji = await this._announcementRepository.GetUserReactionAsync(announcementId, userId);

    if (existingEmoji == emoji)
    {
        await this._announcementRepository.RemoveReactionAsync(announcementId, userId);
    }
    else
    {
        await this.AddOrUpdateReactAsync(announcementId, userId, emoji);
    }
}
```

```
8 references
public async Task AddOrUpdateReactAsync(int announcementId, Guid userId, string emoji)
{
    var existingEmoji = await this._announcementRepository.GetUserReactionAsync(announcementId, userId);

    if (existingEmoji == null)
    {
        await this._announcementRepository.InsertReactionAsync(announcementId, userId, emoji);
        return;
    }

    if (existingEmoji == emoji)
    {
        await this._announcementRepository.RemoveReactionAsync(announcementId, userId);
        return;
    }

    await this._announcementRepository.UpdateReactionAsync(announcementId, userId, emoji);
}
```

Repository:

```
14 references
public async Task RemoveReactionAsync(int announcementId, Guid userId)
{
    using var db = await _contextFactory.CreateDbContextAsync();
    var announcementReaction=await db.AnnouncementReactions
        .Where(ar => ar.AnnouncementId == announcementId && ar.AuthorId == userId)
        .FirstOrDefaultAsync();
    if (announcementReaction != null)
    {
        db.AnnouncementReactions.Remove(announcementReaction);
        await db.SaveChangesAsync();
    }
}
```

```
6 references
public async Task InsertReactionAsync(int announcementId, Guid userId, string emoji)
{
    using var db = await _contextFactory.CreateDbContextAsync();
    var announcement = await db.Announcements.FindAsync(announcementId);
    if (announcement == null)
    {
        throw new InvalidOperationException("Announcement not found");
    }

    var user = await db.Users.FindAsync(userId);
    if (user == null)
    {
        throw new InvalidOperationException("User not found");
    }

    var announcementReaction = new AnnouncementReaction
    {
        AnnouncementId = announcementId,
        AuthorId = userId,
        Author = user,
        Emoji = emoji
    };

    db.AnnouncementReactions.Add(announcementReaction);
    await db.SaveChangesAsync();
}
```